UNIVERSIDADE FEDERAL DE SERGIPE

Lista duplamente encadeada

Um nó que sabe quem vem antes — e exercícios

Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe
andreyoshiaki@dcomp.ufs.br

O que você vai aprender hoje

- O nó com duas setas: `ant` e `prox`
- Inserir: quatro ponteiros, nenhum esquecido
- Remover dado o nó, sem procurar o anterior
- Nó sentinela: inserir e remover sem nenhum `if`
- Quanto custa: tempo e memória
- Exercícios

A pergunta de hoje

O problema: a lista só anda para a frente

Até aqui

Na `lista.c`, para tirar um nó era preciso andar com `ant` um passo atrás, porque só o anterior sabe onde o nó está. Tirar o *último* custava a lista inteira — e imprimir de trás para a frente, nem pensar.

O que vamos descobrir

Basta cada nó guardar também **quem vem antes**. O preço: um ponteiro a mais por nó, e o dobro de setas para manter certas a cada inserção e remoção.

A pergunta de hoje

Como remover um nó tendo só o próprio nó — e o que é preciso fazer para que as setas de ida e de volta nunca discordem?

O programa de hoje: `dupla.c` (1/3)

```c
/* dupla.c --- lista duplamente
   encadeada em ordem crescente */
#include <stdio.h>
#include <stdlib.h>

typedef struct no {
    int valor;
    struct no *ant;   /* ou NULL */
    struct no *prox;  /* ou NULL */
} No;

typedef struct {
    No *ini;   /* primeiro, ou NULL */
    No *fim;   /* ultimo, ou NULL */
} Lista;
```

```c
void imprime_volta(const Lista *l)
{
    printf("volta:");
    for (const No *p = l->fim; p != NULL;
         p = p->ant)
        printf(" %d", p->valor);
    printf("\n");
}
```

`busca`, `imprime` e `libera` são as da aula passada, recebendo um `Lista *`. `imprime_volta` faz o caminho contrário: começa no `fim` e segue os `ant`.

dupla.c (2/3): inserir em ordem e remover

```c
void insere_ordenado(Lista *l, int x)
{
    No *novo = malloc(sizeof *novo);
    if (novo == NULL) { perror("malloc"); exit(1); }
    novo->valor = x;
    No *p = l->ini;   /* novo vai antes de p */
    while (p != NULL && p->valor < x)
        p = p->prox;
    novo->prox = p;
    novo->ant = (p == NULL) ? l->fim : p->ant;
    if (novo->ant == NULL) l->ini = novo;
    else novo->ant->prox = novo;
    if (p == NULL) l->fim = novo;
    else p->ant = novo;
}
```

```c
/* tira o no p, sem procurar
   o anterior */
void remove_no(Lista *l, No *p)
{
    if (p->ant == NULL)
        l->ini = p->prox;
    else
        p->ant->prox = p->prox;
    if (p->prox == NULL)
        l->fim = p->ant;
    else
        p->prox->ant = p->ant;
    free(p);
}
```

dupla.c (3/3): o main

```c
int main(void)
{
    int entrada[] = {30, 10, 40, 20};
    Lista l = {NULL, NULL};
    for (int i = 0; i < 4; i++)
        insere_ordenado(&l, entrada[i]);
    imprime(&l);
    imprime_volta(&l);
    No *p = busca(&l, 20);
    if (p != NULL)
        remove_no(&l, p);
    remove_no(&l, l.fim);
    imprime(&l);
    imprime_volta(&l);
    libera(&l);
    return 0;
}
```

Os valores chegam fora de ordem: 30, 10, 40, 20 — os mesmos da aula passada.

Depois, o programa tira o 20, achado pela busca, e o último nó, que já está à mão em `l.fim`.

O que este programa imprime?

Antes de compilar, escreva sua resposta

1. Quais são as duas primeiras linhas, `ida:` e `volta:`?
2. Qual nó é `l.fim` na hora do segundo `remove_no`? E quais são as duas últimas linhas?
3. Em cada `remove_no`, quais dos dois `if` vão para o `else`?
4. Quantos nós cada `remove_no` percorre para achar o anterior?

Compile, execute e compare

```
$ gcc -Wall -Wextra dupla.c -o dupla
$ ./dupla
ida:   10 20 30 40
volta: 40 30 20 10
ida:   10 30
volta: 30 10
```

O que mudou

A lista agora se lê **nos dois sentidos**, e as duas leituras concordam — antes e depois das remoções.

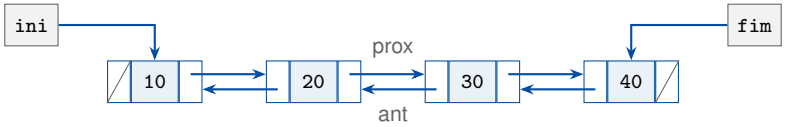
Nenhuma das remoções percorreu a lista: o nó sabia quem era o seu anterior. O 20 caiu nos dois `else`; o 40, só no primeiro.

O nó com duas setas

Definição: lista duplamente encadeada

Lista duplamente encadeada

Cada nó guarda, além do valor, um ponteiro `prox` para o seguinte e um ponteiro `ant` para o anterior. O primeiro tem `ant == NULL`; o último, `prox == NULL`.



A “lista” passa a ser um registro com **dois** ponteiros: `ini` e `fim`.

Fonte: Cormen et al. (2009), §10.2 [2]; Celes et al. (2004), cap. 10 [1]

A regra que ninguém confere por você

Consistência

Para todo nó `p` que tem seguinte:

```
p->prox->ant == p
```

quem eu aponto para a frente aponta de volta para mim. E mais: `ini->ant` e `fim->prox` são `NULL`.

Cada operação precisa *preservar* essa regra — o compilador não sabe que ela existe.

O preço em memória

nó	`sizeof`
simples (`valor`, `prox`)	16 bytes
duplo (`valor`, `ant`, `prox`)	24 bytes

`int` = 4, ponteiro = 8, mais 4 bytes de alinhamento antes do primeiro ponteiro (64 bits).

Fonte: Knuth (1997), §2.2.5 [5]; ISO/IEC 9899:2018, §6.7.2.1 [4]; medido com `tamanho.c` do tutorial

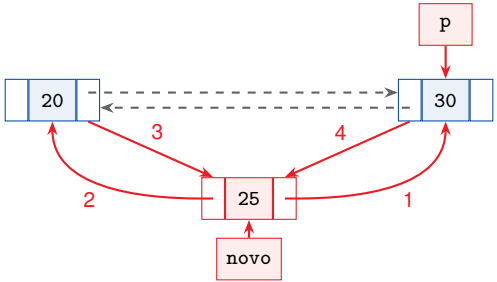
Recapitulando: o nó com duas setas

O que aprendemos

- cada nó tem `ant` e `prox`; a lista guarda `ini` e `fim`;
- dá para percorrer nos dois sentidos, começando por qualquer extremo;
- a regra `p->prox->ant == p` tem de valer depois de *toda* operação;
- o preço: 8 bytes a mais por nó (16 $\rightarrow$ 24).

Inserir: quatro ponteiros

Encaixar o 25 antes do 30




```
1. novo->prox = p;
2. novo->ant = p->ant;
3. novo->ant->prox = novo;
4. p->ant = novo;
```

1 e 2 só *leem* os vizinhos; 3 e 4 os *alteram*. Depois do passo 4, o antigo anterior de `p` já não é alcançável por ele — por isso 2 vem antes de 4.

Fonte: Cormen et al. (2009), §10.2, LIST-INSERT [2]; Celes et al. (2004), cap. 10 [1]

Os extremos trocam um vizinho por `ini` ou `fim`

onde entra o novo	passo 3 vira ...	passo 4 vira ...
no meio	`novo->ant->prox = novo`	`p->ant = novo`
no início	`l->ini = novo`	`p->ant = novo`
no fim (`p == NULL`)	`novo->ant->prox = novo`	`l->fim = novo`
na lista vazia	`l->ini = novo`	`l->fim = novo`

Os dois `if` de `insere_ordenado` são exatamente essas duas colunas. No fim, não existe `p` para dar o anterior: ele vem de `l->fim`.

Fonte: Celes et al. (2004), cap. 10 [1]

E se uma linha faltar?

```
    /* insere_errada.c: o final de
       insere_ordenado */
        novo->prox = p;
        novo->ant = (p == NULL) ? l->fim
                                : p->ant;
        if (novo->ant == NULL)
            l->ini = novo;
        else
            novo->ant->prox = novo;
        if (p == NULL)
            l->fim = novo;
        /* sumiu: else p->ant = novo; */
    }
```

Pense antes de passar o slide

O `main` é o mesmo: insere 30, 10, 40, 20.

1. A linha `ida:` sai certa? E a `volta:`?
2. Depois de inserir o 10, quanto vale o `ant` do 30?
3. Quando o 20 chega, o que a função conclui ao ler `p->ant`?

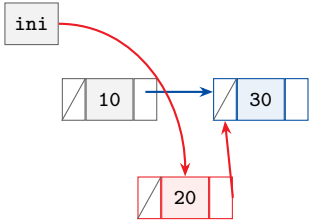
Execute e compare

```
$ gcc -Wall -Wextra insere_errada.c -o insere_errada
$ ./insere_errada
ida:   20 30 40
volta: 40 30
ida:   30
volta: 30
```

Nenhum aviso do compilador, nenhuma queda.

Mas a ida perdeu o 10 — e não foi só a volta que saiu errada.

A resposta: um `ant` errado engana a inserção seguinte



Ao inserir o 10 antes do 30, o `ant` do 30 ficou `NULL`. A volta para no 30 por isso.

Quando o 20 entra antes do 30, a função **usa** `p->ant` para achar o anterior, lê `NULL` e faz `l->ini = novo`. O 10 fica inacessível — e vaza.

Na lista dupla, os `ant` não são enfeite: um `ant` errado corrompe as operações *seguintes*.

Fonte: Knuth (1997), §2.2.5 [5]; medido com `insere_errada.c`

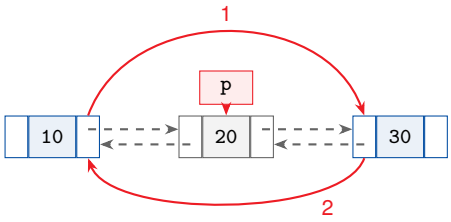
Recapitulando: inserir

O que aprendemos

- inserir antes de `p` muda **quatro** ponteiros: dois do novo, dois dos vizinhos;
- primeiro ler os vizinhos, depois alterá-los;
- no início, o vizinho da esquerda vira `l->ini`; no fim, o da direita vira `l->fim`;
- esquecer um ponteiro pode não dar erro nenhum — só uma lista que mente.

Remover dado o nó

Remover: ligar os dois vizinhos entre si



1. `p->ant->prox = p->prox;`
2. `p->prox->ant = p->ant;`
3. `free(p);`

Na lista simples, só o anterior sabia onde estava o nó — por isso a busca andava com `ant` um passo atrás.

Agora o próprio nó sabe: `p->ant`. Remover não tem **laço nenhum**, e o `free` continua por último.

Fonte: Cormen et al. (2009), §10.2, LIST-DELETE [2]; Celes et al. (2004), cap. 10 [1]

Quatro situações, dois `if`

quem sai	passo 1 vira ...	passo 2 vira ...
um nó do meio	`p->ant->prox = p->prox`	`p->prox->ant = p->ant`
o primeiro	`l->ini = p->prox`	`p->prox->ant = p->ant`
o último	`p->ant->prox = p->prox`	`l->fim = p->ant`
o único	`l->ini = p->prox`	`l->fim = p->ant`

A mesma tabela da inserção, ao contrário. No caso do único nó, os dois extremos recebem `NULL` e a lista fica vazia, como deve.

Fonte: Celes et al. (2004), cap. 10 [1]

Um `remove_no` mais curto

```c
/* remove_errada.c */
void remove_no(Lista *l, No *p)
{
    if (p->ant == NULL)
        l->ini = p->prox;
    else
        p->ant->prox = p->prox;
    if (p->prox != NULL)
        p->prox->ant = p->ant;
    free(p);
}
```

Pense antes de passar o slide

O `else` do segundo `if` sumiu.

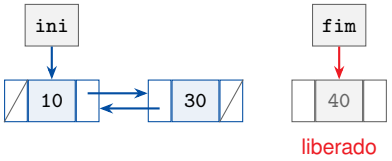
1. Remover o 20 (do meio) funciona?
2. E `remove_no(&l, l.fim)`? Para onde `l.fim` aponta depois?
3. O que o último `imprime_volta` vai ler?

Execute e compare

```
$ ./remove_errada
ida:   10 20 30 40
volta: 40 30 20 10
ida:   10 30
Segmentation fault: 11
$ gcc -g -fsanitize=address remove_errada.c -o remove_errada-asan
$ ./remove_errada-asan
...
==38375==ERROR: AddressSanitizer: heap-use-after-free on address 0x603000001c60
READ of size 4 at 0x603000001c60 thread T0
    #0 0x00010027d22c in imprime_volta remove_errada.c:73
0x603000001c60 is located 0 bytes inside of 24-byte region
freed by thread T0 here:
    #1 0x00010027d018 in remove_no remove_errada.c:58
```

A linha 58 é o `free(p)`; a 73 lê `p->valor` na volta. Saída do relatório resumida.

A resposta: `fim` ficou apontando para um bloco liberado



O 30 passou a ser o último (`prox` = `NULL`), mas `l.fim` ainda guarda o endereço do 40 — um ponteiro **pendente**.

Ler por ele é comportamento indefinido: pode imprimir lixo, pode cair. O ASan mostra as três pontas: onde foi *alocado*, onde foi *liberado* e onde foi *lido*.

Fonte: ISO/IEC 9899:2018, §6.2.4 e §7.22.3.3 [4]; Serebryany et al. (2012) [6]

Recapitulando: remover dado o nó

O que aprendemos

- remover é ligar `p->ant` e `p->prox` entre si e só então chamar `free(p)`;
- não há laço: o custo não depende do tamanho da lista;
- o primeiro atualiza `l->ini`; o último, `l->fim` — são dois `if` independentes;
- um extremo esquecido vira ponteiro pendente: `-fsanitize=address` acusa.

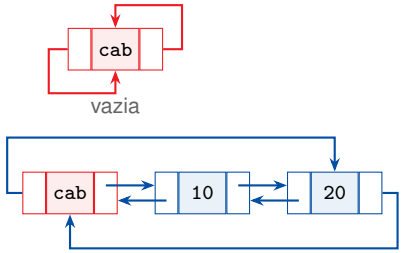
O nó sentinela

E se todo nó tivesse anterior e seguinte?

Os quatro `if` vêm do mesmo lugar: o primeiro não tem anterior, o último não tem seguinte, e a seta que muda mora fora dos nós.

Nó sentinela (cabeça)

Um nó extra, sem valor, que fecha a lista num círculo: `cab->prox` é o primeiro e `cab->ant`, o último. A lista vazia é a cabeça apontando para si mesma.



Fonte: Cormen et al. (2009), §10.2, *sentinels* [2]; Knuth (1997), §2.2.4–2.2.5 [5]

sentinela.c: nenhum if

```c
void inicia(No *cab)
{
    cab->ant = cab->prox = cab;
}

void insere_antes(No *p, int x)
{
    No *novo = malloc(sizeof *novo);
    if (novo == NULL) { perror("malloc"); exit(1); }
    novo->valor = x;
    novo->prox = p;
    novo->ant = p->ant;
    p->ant->prox = novo;
    p->ant = novo;
}
```

```c
void remove_no(No *p)
{
    p->ant->prox = p->prox;
    p->prox->ant = p->ant;
    free(p);
}

void insere_ordenado(No *cab, int x)
{
    No *p = cab->prox;
    while (p != cab && p->valor < x)
        p = p->prox;
    insere_antes(p, x);
}
```

Pense antes de passar o slide

Leia o código com calma

1. Na lista vazia, `insere_ordenado` chama `insere_antes(cab, x)`. Siga as quatro linhas: como fica a lista?
2. Por que o laço para em `p != cab`, e não em `p != NULL`?
3. Como se insere no *fim*, sem laço? E como se remove o último?
4. A cabeça é `No cab;` no `main`. Ela pode ir para o `free`?

A resposta: a cabeça faz o papel de `ini`, `fim` e `NULL`

- na lista vazia, `p->ant` é a própria cabeça: o novo passa a ser o anterior *e* o seguinte dela;
- não existe `NULL` no círculo: o fim do percurso é voltar à cabeça;
- o fim é "antes da cabeça": `insere_antes(&cab, x);` o último é `cab.ant`: `remove_no(cab.ant);`
- a cabeça não veio do `malloc`: `free(&cab)` é comportamento indefinido.

O mesmo truque do `No **`

Na aula passada, `No **` apontava para "a seta que muda", fosse ela `lista` ou um `prox`. A cabeça vai além: transforma `ini` e `fim` em campos de um nó — e o caso especial some.

A saída de `sentinela.c` é idêntica à de `dupla.c`.

Fonte: Cormen et al. (2009), §10.2 [2]; ISO/IEC 9899:2018, §7.22.3.3 [4]

Recapitulando: o nó sentinela

O que aprendemos

- uma cabeça sem valor fecha a lista num círculo: todo nó tem anterior e seguinte;
- inserir e remover viram quatro e duas linhas, sem `if`;
- percorrer é ir de `cab->prox` até voltar a `cab`;
- o preço: um nó a mais, e cuidado para não tratar a cabeça como um valor.

Quanto custa

Remover sempre o último: simples $\times$ dupla

n	simples	dupla
10 000	0,034 s	0,0001 s
20 000	0,189 s	0,0002 s
40 000	0,678 s	0,0005 s
80 000	2,689 s	0,0010 s
160 000	9,838 s	0,0019 s

monta n nós e remove o último até esvaziar; `-O2`

Na simples, cada remoção procura o penúltimo: $(n-1)+\cdots+1 \approx n^2/2$ passos. Dobrou n, **quadruplicou** o tempo.

Na dupla, o penúltimo é `fim->ant`: dobrou n, dobrou o tempo — um custo fixo por remoção.

Guardar `fim` na simples não resolve: depois de remover, quem é o novo `fim`?

Fonte: Cormen et al. (2009), §10.2 [2]; medido com `remove_fim.c`

O que cada lista faz sem percorrer

operação	simples (`ini`)	simples (`ini`, `fim`)	dupla
inserir no início	constante	constante	constante
inserir no fim	n	constante	constante
remover o primeiro	constante	constante	constante
remover o último	n	n	constante
remover um nó dado	n	n	constante
buscar um valor	n	n	n
memória por nó (`int`)	16 B	16 B	24 B

A busca não melhora: para achar um valor, ainda é preciso olhar os nós um a um. A dupla compensa quando o programa **já tem o nó** na mão — ou precisa andar para trás.

Fonte: Cormen et al. (2009), §10.2 [2]; Tenenbaum et al. (1995), cap. 4 [7]

Recapitulando: quanto custa

O que aprendemos

- remover o último ou um nó dado passa de n passos para um custo fixo;
- buscar continua custando n: o `ant` não ajuda a achar;
- o preço é 50% a mais de memória por nó (com `int`) e o dobro de setas para manter;
- escolha a dupla quando o programa guarda ponteiros para nós ou anda nos dois sentidos.

Mãos à obra

Altere o programa

Tarefa 1

Troque as duas remoções do `main` por um laço que remove alternadamente `l.ini` e `l.fim` até esvaziar, imprimindo ida e volta a cada passo. Que linhas de `remove_no` rodam na última remoção?

Tarefa 2

Escreva `busca_volta`, que começa em `l->fim`. Numa lista em ordem crescente, como escolher entre `busca` e `busca_volta` sem percorrer nada?

Tarefa 3

Insira o 20 duas vezes. O novo entra antes ou depois do antigo? Troque < por <= e preveja o que muda — confira com `%p`.

Exercícios (1/3)

Exercício 1 — a lista está consistente?

`int consistente(const Lista *l)`: devolve 1 se valem `ini->ant == NULL`, `fim->prox == NULL`, `p->prox->ant == p` para todo `p`, e se andar pelos `prox` termina em `fim`. Use-a para achar o erro de `insere_errada.c`.

Exercício 2 — inserir nos extremos

`insere_inicio(Lista *l, int x)` e `insere_fim`, sem laço. Qual dos quatro ponteiros vira `l->ini` na lista vazia?

Exercício 3 — remover todas as ocorrências

`remove_todos(Lista *l, int x)` usando `remove_no`. Por que é preciso guardar `p->prox` *antes* de remover?

Fonte: Celes et al. (2004), cap. 10 [1]; Feofiloff (2009), cap. 4 [3]

Exercícios (2/3)

Exercício 4 — inverter com sentinela

`void inverte(No *cab)`: troque `ant` e `prox` em todos os nós, inclusive na cabeça. Por que a cabeça também? Quanto custa fazer o mesmo sem sentinela?

Exercício 5 — concatenar

`void concatena(Lista *a, Lista *b)` pendura os nós de `b` no fim de `a`, sem `malloc` e sem laço, e deixa `b` vazia. Não esqueça de `a` ou `b` vazias.

Exercício 6 — palíndromo

`int palindromo(const Lista *l)`: dois ponteiros, um saindo de `ini` e outro de `fim`, andando um em direção ao outro. Quando eles param? Cuidado com n par e ímpar.

Fonte: Knuth (1997), §2.2.5 [5]; Tenenbaum et al. (1995), cap. 4 [7]

Exercícios (3/3)

Exercício 7 — mover para a frente

`void para_frente(Lista *l, No *p)` move o nó `p` para o início, sem `malloc` nem `free`. Quantos ponteiros mudam? Compare com o Exercício 4 da aula passada.

Exercício 8 — de simples para dupla

Dada uma lista em que só os `prox` estão certos (os `ant` têm lixo), preencha todos os `ant` e o `fim` numa única passada.

Exercício 9 — girar

Numa lista com sentinela, `void gira(No *cab, int k)` faz os k primeiros irem para o fim (`10 20 30 40`, $k = 1 \rightarrow$ `20 30 40 10`). Dá para fazer movendo só a cabeça?

Fonte: Cormen et al. (2009), §10.2, exercícios [2]; Feofiloff (2009), cap. 4 [3]

Desafio

Um editor de linha com cursor

Os caracteres da linha ficam numa lista dupla circular com sentinela; o cursor é um ponteiro para o nó *à direita* dele (a cabeça, se está no fim). Leia comandos até o fim da entrada:

- `i texto` insere os caracteres à esquerda do cursor;
- < e > movem o cursor (<<< move três vezes), sem passar das pontas;
- `x` apaga o caractere à esquerda (*backspace*); = imprime a linha com | no cursor.

Nenhum comando pode percorrer a linha. Com `-fsanitize=address`, nenhum erro.

`i progama` → `progama|` <<< `i r` → `progr|ama`

Fonte: Knuth (1997), §2.2.5 [5]; Cormen et al. (2009), §10.2 [2]

Fechamento

O que mudou hoje

Com `ant` em cada nó, a lista anda nos dois sentidos e um nó sai dela sem que ninguém precise procurá-lo.

O que ficou

Inserir e remover nas pontas ficou barato. Uma estrutura que *só* faz isso já tem nome.

Para levar

- regra: `p->prox->ant == p` depois de toda operação;
- inserir antes de `p`: ler os vizinhos, depois mudar `ant->prox` e `p->ant`;
- remover: `p->ant->prox = p->prox; p->prox->ant = p->ant; free(p);`
- extremos são especiais — a menos que haja uma cabeça.

Próxima aula: pilhas — só se põe e se tira por uma das pontas.

Referências I

- Waldemar Celes, Renato Cerqueira, and José Lucas Rangel.
  Introdução a estruturas de dados: com técnicas de programação em C.
  Elsevier, Rio de Janeiro, 2004.
  Capítulo 10: listas encadeadas (listas circulares e duplamente encadeadas).
- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
  Introduction to Algorithms.
  MIT Press, Cambridge, MA, 3 edition, 2009.
  Seção 10.2: linked lists (listas duplamente encadeadas e sentinelas).

Referências II



Paulo Feofiloff.
Algoritmos em linguagem C.
Elsevier, Rio de Janeiro, 2009.
Capítulo 4: listas encadeadas.



ISO/IEC.
ISO/IEC 9899:2018 — Information technology: Programming languages: C.
Technical report, International Organization for Standardization, 2018.
Seções 6.2.4 (tempo de vida), 6.7.2.1 (preenchimento em estruturas) e 7.22.3.3 (`free`).

Referências III


Donald E. Knuth.
The Art of Computer Programming, Volume 1: Fundamental Algorithms.
Addison-Wesley, Reading, MA, 3 edition, 1997.
Seções 2.2.4 (listas circulares) e 2.2.5 (listas duplamente encadeadas).


Konstantin Serebryany, Derek Bruening, Alexander Potapenko, and Dmitriy Vyukov.
AddressSanitizer: A fast address sanity checker.
In *USENIX Annual Technical Conference (ATC)*, pages 309–318, 2012.

Referências IV


Aaron M. Tenenbaum, Yedidyah Langsam, and Moshe J. Augenstein.
Estruturas de dados usando C.
Pearson Makron Books, São Paulo, 1995.
Capítulo 4: filas e listas (listas duplamente ligadas).